Solving the Multiple-Sequence Variable-Gapped Longest Common Subsequence Problem

Marko Djukanović[1,2,6], Nikola Balaban[2], Christian Blum[3], Aleksandar Kartelj[4], Sašo Džeroski[5], and Žiga Zebec[6]

[1] University of Nova Gorica, Nova Gorica, Slovenia
marko.dukanovic@ung.si
[2] Faculty of Natural Sciences and Mathematics, University of Banja Luka, Banja Luka, Bosnia and Herzegovina
nikola.balaban@student.pmf.unibl.org, marko.djukanovic@pmf.unibl.org
[3] Artificial Intelligence Research Institute (IIIA-CSIC), Barcelona, Spain
christian.blum@iiia.csic.es
[4] Faculty of Mathematics, University of Belgrade, Belgrade, Serbia
kartelj@matf.rs
[5] Jožef Stefan Institute, Ljubljana, Slovenia
saso.dzeroski@ijs.si
[6] Institute of Information Sciences (IZUM), Maribor, Slovenia
ziga.zebec@izum.si

Abstract. This paper addresses the Variable-Gapped Longest Common Subsequence Problem (VGLCSP), a generalization of the classical LCS problem in which flexible gap constraints are imposed between consecutive characters of a solution. The problem arises in molecular sequence comparison, where structural distance constraints between residues must be respected, and in time-series analysis, where events must occur within prescribed temporal delays. We propose a search framework based on a rooted state-graph representation, in which the state space consists of a potentially large number of rooted subgraphs. To control the resulting combinatorial growth, we develop an Iterative Multi-Source Beam Search (IMSBS) that maintains a global pool of promising candidate roots and dynamically balances intensification within a rooted subgraph against diversification across subgraphs. The beam-search component incorporates several heuristic estimates adapted from the LCS literature. To the best of our knowledge, this is the first comprehensive computational study of the VGLCSP with more than two input sequences. The study comprises 320 synthetic instances with up to 10 sequences of length up to 500. The results show that IMSBS is more robust than a baseline beam search while requiring comparable runtimes.

Keywords: Longest common subsequence · Beam search · Bioinformatics · Gap constraints

1 Introduction

The *Longest Common Subsequence Problem* (LCSP) [6, 3] is a well-known combinatorial optimization problem with numerous applications in bioinformatics and

computational biology, where it plays a fundamental role in the analysis and comparison of molecular sequences. Given a set of input sequences $S = \{s_1, \ldots, s_m\}$, with $m \in \mathbb{N}$, over an alphabet Σ, the aim is to identify a longest subsequence common to all sequences $s_i \in S$. Over the past decades, a variety of practically motivated extensions of the LCSP have been proposed to better capture structural, biological, or application-specific requirements of real-world sequences. Notable variants include constrained, arc-preserving, and repetition-free longest common subsequence problems [8, 7, 2], among others. In this work, we focus on another practically motivated variant, the *Variable-Gapped LCS Problem* (VGLCSP), originally introduced for the $m = 2$ case in [11] and later investigated from a theoretical perspective in [1]. The VGLCSP extends the classical LCSP by incorporating flexible distance constraints (referred to as *gaps*) between consecutive symbols of the resulting subsequences and their respective positions in the input sequences. Unlike fixed-gap models, these gap limits are allowed to vary along the input sequences, thereby offering increased modeling flexibility.
Formally, for each input sequence $s_i \in S$, its gap constraint is defined as a function over the positions of the sequence, i.e., $G_i \colon \{1, \ldots, |s_i|\} \mapsto \mathbb{N}$. Further, if the characters of a solution s occur at positions $i_i^1 < \cdots < i_i^{|s|}$ in s_i, the gap constraint is satisfied if and only if $i_i^x - i_i^{x-1} \leq G_i[i_i^x] + 1, \quad x = 2, \ldots, |s|$.
A common subsequence s is considered *feasible* if the corresponding gap constraints are satisfied simultaneously for all sequences in S. For example, consider $S = \{s_1 = \texttt{ABCA}, s_2 = \texttt{ACAB}\}$ with constant gap constraints $G_1(j) = G_2(j) = 1$ for all valid positions j. The sequence $s = \texttt{ACA}$ is a feasible solution because it is a common subsequence for both input strings, where its corresponding characters appear at positions 1, 3, and 4 in s_1 and at positions 1, 2, and 3 in s_2. Because the difference between each pair of consecutive indices is less than or equal to $1 + 1 = 2$, s is feasible. By contrast, $s = \texttt{AB}$ is a common subsequence but is infeasible because the corresponding characters appear at positions 1 and 2 in s_1 and at positions 1 and 4 in s_2. Indeed, $4 - 1 = 3 > 1 + 1 = 2$, which violates the gap constraint in s_2.
The VGLCSP provides a flexible and realistic model for sequence comparison, particularly suitable for applications in DNA and protein analysis where variable structural distances between (active) residues must be respected. Beyond bioinformatics, the problem also arises in time-series analysis, especially in settings where events are required to occur within specified temporal delays [9].
Although several dynamic-programming approaches exist for the $m = 2$ case [11], they are computationally prohibitive when extended to larger values of m. To the best of our knowledge, no effective exact or approximate approaches have been proposed for the generalized VGLCSP with arbitrary m, an $\mathcal{NP}$-hard problem because it generalizes the LCS problem with an arbitrarily large set of input strings.

Preliminaries. Let $|s|$ denote the length of a sequence s, and let $s[i]$ refer to the character of s at position i, where indexing starts at $i = 1$. The substring of s that begins at position i and ends at position j is denoted by $s[i, j]$. If

$i = j$, this corresponds to the single character $s[i]$; if $j < i$, then $s[i, j] = \varepsilon$, where ε denotes the empty string. The number of occurrences of $a \in \Sigma$ in s is denoted by $|s|_a$. We denote by $S = \{s_1, \ldots, s_m\}$ the set of input sequences, and by $S^{rev} = \{s_1^{rev}, \ldots, s_m^{rev}\}$, the set of reverse strings. Let $m \in \mathbb{N}$ be the number of input sequences (and, correspondingly, the number of gap constraints), and let n denote the length of the longest sequence in S. Given a positional vector $\mathbf{p}^L \in \mathbb{N}^m$, the subproblem related to these positions is given by $S[\mathbf{p}^L] = \{s_i[\mathbf{p}_i^L, |s_i|] \mid i = 1, \ldots, m\}$. Finally, for a gap constraint G_i, its reverse constraint is denoted by G_i^{rev}, given by $G_i^{rev}(j) := G_i(|s_i| - j + 1)$, for each $j = 1, \ldots, |s_i|$.

The remainder of the paper is organized as follows. Section 2 introduces the rooted graph representation of the problem. In Section 3, we design our main methodological contribution, the Iterative Multi-Source Beam Search approach. Section 4 provides a mathematical formulation for the problem with two input sequences as an alternative to the common dynamic-programming approaches. Section 5 reports the computational results, comparing the proposed method with a baseline beam search and an iterative greedy heuristic. Additionally, all literature approaches designed for two input sequences are implemented and compared to the introduced model. Finally, Section 6 concludes the paper and outlines directions for future research.

2 Rooted State Space Formulation

Motivated by the state-space representation for the LCSP introduced in [6], we design a *rooted state graph* model for the VGLCSP. Each state represents one or more feasible partial solutions characterized by a vector of positions induced by the input sequences and by the current subsequence length.

Formally, a partial subsequence s^v induces a state node $v = (\mathbf{p}^{L,v}, l^v)$, where $\mathbf{p}^{L,v} = (p_1^{L,v}, \ldots, p_m^{L,v})$ and the following conditions hold: (i) for each sequence $s_i \in S$, $p_i^{L,v}-1$ is the smallest position such that s^v is a subsequence of $s_i[1, p_i^{L,v}-1]$; (ii) $l^v = |s^v|$ denotes the length of the partial subsequence; and (iii) all gap constraints G_i are *satisfied* for s^v with respect to each input sequence s_i. A directed arc $\alpha = (v_1, v_2)$, labeled with $\texttt{lett}(\alpha) = a \in \Sigma$, exists if $l^{v_2} = l^{v_1} + 1$ and $s^{v_2} = s^{v_1} \cdot a$; that is, state v_2 corresponds to extending a partial solution of v_1 by appending character a, respecting all gap constraints.

To expand a state v, those characters $a \in \Sigma$ that occur in each suffix $s_i[p_i^{L,v}, |s_i|]$, $i = 1, \ldots, m$, are first detected. For each such $a \in \Sigma$, we identify the smallest feasible positions $p_i^{L,a} \geq p_i^{L,v}$ such that $s_i[p_i^{L,a}] = a$ and $p_i^{L,a} - p_i^{L,v} \leq G_i(p_i^{L,a}), i = 1, \ldots, m$. In that way, a child state $w = (\mathbf{p}^{L,a} + \mathbf{1}, l^v + 1)$ is created, unless it is dominated by another child state according to the dominance criteria defined later in this section. To efficiently compute children and their associated position vectors, we employ a preprocessing data structure denoted by `Succ`: a three-dimensional integer array indexed by (i, j, a), where i refers to the input sequence s_i, $1 \leq j \leq |s_i|$ denotes a position within that sequence, and $a \in \Sigma$.

The value $q = \texttt{Succ}[i, j, a]$ represents the smallest index $q \geq j$ such that $s_i[q] = a$ so that the gap constraint G_i is satisfied at position q, i.e., $q - j \leq G_i(q)$. If no such position exists, $q = -1$ is assigned. Terminal states are those that cannot be expanded further. The trivial root state $r = ((1, \ldots, 1), 0)$ has no incoming arcs and represents the empty partial solution, as in the classical LCS problem. In general, however, it is not the only root state. The state graph may contain many, possibly exponentially many, root states. Selecting which root states to explore is therefore a central algorithmic decision. Note that if match $u = (i_1^1, \ldots, i_m^1)$ represents a starting position, the gap constraints do not apply to the corresponding letter of this or any other rooted match.

This motivates the introduction of a state-space subgraph rooted at match $\mathbf{p}^{L,u}$, denoted by RootedSubgraph$(u = (\mathbf{p}^{L,u}, 0))$, where u is associated with the empty partial solution. The empty solution is first extended by the character l matched at $\mathbf{p}^{L,u}$, thereby generating all feasible solutions whose leading character is l. This subgraph consists of all states, together with their transitions, that are feasibly reachable from $(\mathbf{p}^{L,u} + \mathbf{1}, 1)$. These states correspond to partial solutions whose leading character is matched at position vector $\mathbf{p}^{L,u}$. The expansion procedure is applied recursively until no feasible extension remains. As discussed in the following section, the structure of this subgraph depends strongly on the choice of the root state u from which the search begins. Figure 1 illustrates RootedSubgraph$(u = ((1, 1), 0))$. Here, position $(1, 1)$ corresponds to a match of the character A associated with the root state $u = ((1, 1), 0)$.

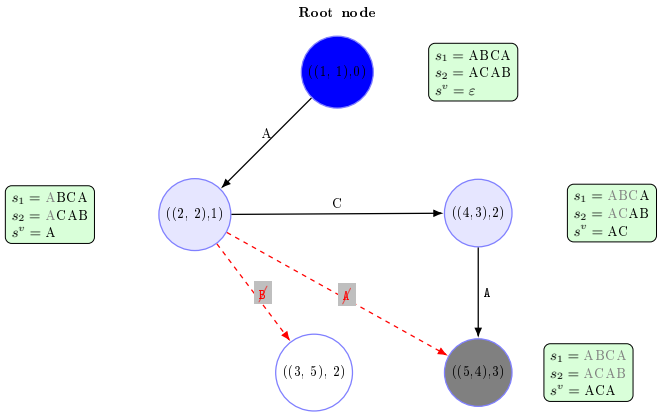


Fig. 1: RootedSubgraph$(u = ((1, 1), 0))$ for A BCA and A CAB, assuming $G_1 = G_2 = \mathbf{1}$. A final/terminal state is displayed in gray. The white-filled node is a valid LCS node, but not a VGLCS node—the second gap constraint is violated. The longest solution generated from this subspace is ACA.

Multi-Source Beam Search. Unlike the classical LCSP, the VGLCSP may exhibit multiple, potentially exponentially many, *disconnected root components*

in the full state space. Consequently, a search strategy that expands states from a single initial root node may fail to reach optimal solutions.

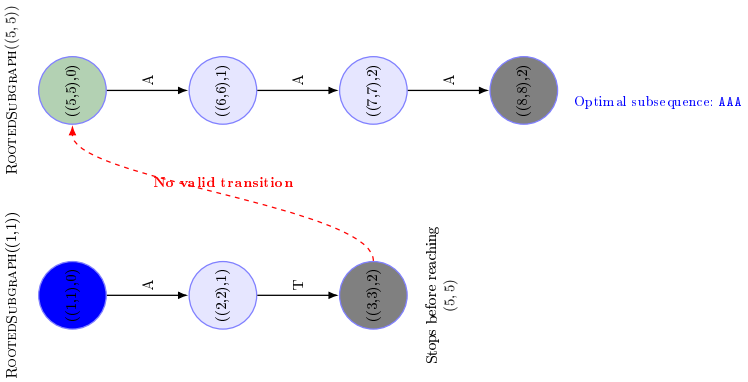


Fig. 2: Disconnected rooted subgraphs of the state space.

As an illustrative example given in Fig. 2, consider the sequence set $S = \{s_1 = \texttt{ATGGAAA},\ s_2 = \texttt{ATCCAAA}\}$, with $G_{s_1} = G_{s_2} = 1$. In this instance, any state with position vector $\mathbf{p}^L = (5, 5)$ is not reachable from the initial state $((1, 1), 0)$ by a series of direct transitions. As a result, the optimal common subsequence `AAA` is unreachable from $((1, 1), 0)$ when exploring its subgraph. To address this issue, we define the complete state space of a VGLCSP instance by $\bigcup_{v\in\mathcal{R}} \textsc{RootedSubgraph}((\mathbf{q}^{L,v}, 0))$, where $\mathcal{R}$ denotes the set of all root states, that cannot be reached from any other state through a sequence of directed transitions. However, explicitly enumerating all such root states is computationally prohibitive and generally requires $O(n^m)$ time, thus remains infeasible for instances with large m or n. This structural property motivates the proposed *Iterative Multi-Source Beam Search* (IMSBS), which dynamically explores multiple promising rooted regions by systematically shifting the search from one subgraph to another. By iteratively identifying and expanding a set of candidate root states, IMSBS effectively mitigates the disconnection effects induced by gap constraints, enabling the discovery of high-quality solutions that would otherwise remain inaccessible to single-source search methods.

3 Iterative Multi-Source Beam Search Framework

Beam search (BS) is a well-established tree-search-based metaheuristic that performs in a breadth-first-search manner. Its search is controlled by a beam width parameter β—specifying the maximum number of nodes retained for expansion at each level—and by heuristic guidance h— which guides the selection of the most promising candidates for further expansion. Initially, the procedure receives

a set of nodes as the initial beam. After being evaluated, up to β most promising nodes are kept for creating a beam of the subsequent level. The procedure repeats until the beam is empty, returning the longest partial solution.

In IMSBS, the beam-search component intensifies the exploration of individual rooted subgraphs. As the search space of a VGLCSP instance is represented by the set of root nodes $\mathcal{R}$, the BS procedure is in charge of efficiently exploring various regions RootedSubgraph$((\mathbf{p}^L, 0))$ induced by corresponding root nodes. Note that one could here employ an exact technique (such as A* or BFS); however, solving the subproblem itself could be a resource-demanding task, as hard as the original problem itself. Thus, we opted to utilize a more robust BS technique.

The core of the IMSBS framework is summarized as follows. Initially, the global pool of candidate root nodes is initialized to an empty set $\mathcal{R} := \{\}$ and the current best solution to $s_{\text{best}} := \varepsilon$. First, for each $a \in \Sigma$, the closest matches $\mathbf{p}^{L,a}$ from the leading positions of each input sequence are detected. Each such (non-dominated) match generates one root node $(\mathbf{p}^{L,a}, 0)$, which has been subsequently added to $\mathcal{R}$. The core iterations of IMSBS have been executed until either $\mathcal{R}$ becomes empty, a predefined maximum number of iterations is reached, or allowed time limit is exceeded. At each iteration, a fixed number (*sources_from_R*) of most promising candidate root nodes according to a heuristic h' is taken from $\mathcal{R}$, and subsequently stored in a temporary beam $\mathcal{L}$. These nodes are refined, executing a single backward beam search procedure for each $w \in \mathcal{L}$ as its starting node $(\mathbf{p}^w, 0)$. The procedure works similarly to the baseline BS, expanding nodes according to the gap constraints but in the reversed manner, starting from the root positions at each sequence and moving the search (pointers) towards their leading positions. In essence, for each $w \in \mathcal{L}$, a beam search is executed on the reversed input sequences S^{rev} and the reversed gap constraints $(G_i^{rev})_{i=1}^m$ with the initial beam width $B = \{((|s_i| - p^w)_{i=1}^m, 0)\}$, beam size β' and heuristic guidance h'. Assume the procedure has completed, returning a solution s_{bwrd}^w. In this way, the reverse sequence s_{bwrd}^{rev} represents a feasible VGLCS solution and the partial solution without the last letter—which corresponds to the match $\mathbf{p}^w$—is assigned to the refined node $w \in \mathcal{L}$, and the updated node $w = (p^w, |(s_{bwrd}^w)^{rev}| - 1)$ is passed to the forward beam search procedure for the forward-pass beam search. Thus, the refined set $\mathcal{L}$ enters in as the initial beam, executing a BS parametrized with β and h, following the subspace definition in Section 2. During this phase, all complete (non-expandable) nodes are stored in the list $V_{complete}$, which is, after termination, returned along with the best found solution s_b. Each complete node from $V_{complete}$ is then independently expanded—ignoring the gap constraints— to generate a set of candidate root nodes. More precisely, for each complete node $v = (\mathbf{p}^v, l^v)$, candidate nodes $r^{w,a} = (\mathbf{p}^{w,a}, 0)$ are generated for each letter $a \in \Sigma$, where $p_i^{w,a}$ represents the match position of that letter in s_i which is closest from position $\mathbf{p}_i^v$. Each such candidate node $(r^{w,a}, 0)$ is added to $\mathcal{R}$, given that it has not already been part of $\mathcal{R}$ during the previous iterations. Once the BS procedure is terminated, the returned solution is checked as a new incumbent, and the next Imsbs iteration

is initiated with a newly selected set of most promising root nodes from $\mathcal{R}$, until a termination condition is met. The pool $\mathcal{R}$ may be maintained as a priority queue implemented via a binary heap, to ensure efficient extraction of the most promising candidate root nodes.
The core advantages of the IMSBS approach over the baseline BS are: (i) A balance between complete beam search execution (local exploration of promising search subregions) and the generation of new candidate root nodes (global coverage of the search space); (ii) The risk of selecting poor candidate root nodes is reduced by executing the backward beam search procedure.
Heuristic guidances. Several heuristic guidances to guide search are utilized, known from the LCS literature: (i) *"Look-ahead" for the remaining length:* $\mathrm{UB}_1(\mathbf{v}) = l^v + \min_{i=1,\dots,m}\left(|s_i| - p_i^{L,v} + 1\right)$; (ii) *Character Frequency Alignment:* $\mathrm{UB}_2(\mathbf{v}) = l^v + \sum_{\sigma\in\Sigma}\min\left(|s_1[p_1^{L,v}, |s_1|]|_\sigma, \dots, |s_m[p_m^{L,v}, |s_m|]|_\sigma\right)$, counting the maximum number of characters that can still match in the (optimal) solution based on the frequency of each character; (iii) *A probability-based heuristic* guidance h_{prob} by the Mousavi and Tabataba's matrices [10]: These probabilities are based on determining a probability of the event realization that a sequence s of length k is a subsequence of a (random) sequence of length n over Σ. Further details on these heuristics can be found in [6, 10].

To conclude, the simplified workflow of the IMSBS approach is provided in Fig. 3.

4 An Integer Linear Programming Model for the VGLCSP with $m = 2$

This section is devoted to deriving an integer linear programming (ILP) model for the polynomially solvable special case of the VGLCSP with two input sequences. Apart from the fact that there are several dynamic-programming approaches in the literature addressing this particular problem, a clear integer-programming formulation does not appear to have been reported. Given that the ILP of some versions of the LCS problem are especially efficient [4], this provides an additional reason for studying the efficiency of such an approach in solving the VGLCSP.

Consider a VGLCSP instance $\mathcal{I} = (s_1, s_2, G_{s_1}, G_{s_2}, \Sigma)$. Let $M = \{(i,j) \mid s_1[i] = s_2[j]\}$ denote the set of all positions where the characters match. For each $(i,j) \in M$, we define a binary variable:

$$x_{i,j} = \begin{cases} 1, & \text{if the pair } (i,j) \text{ is selected as part of the solution,} \\ 0, & \text{otherwise.} \end{cases}$$

We introduce additional binary variables $s_{i,j}$ indicating whether the pair (i,j) represents the *starting position of the resulting subsequence.*

The objective is to maximize the number of selected admissible matches $\max \sum_{(i,j)\in M} x_{i,j}$. The following constraints are imposed:

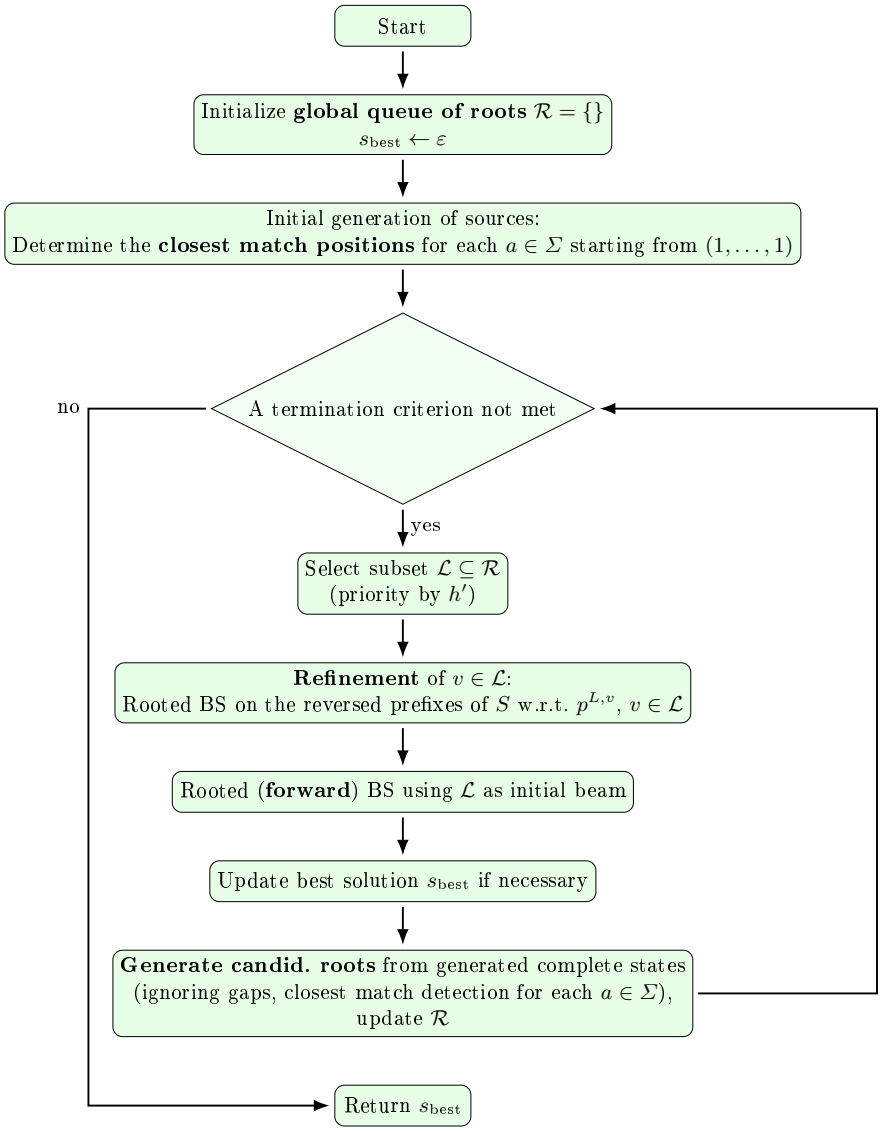


Fig. 3: Simplified workflow of the Iterative Multi-Source Beam Search (IMSBS).

1. Predecessors (variable gap). For each $(i, j) \in M$, we define the set of valid predecessors: $\texttt{Pred}(i, j) = \{(i', j') \in M \mid i' < i,\ j' < j,\ i - i' \leq G_{s_1}[i] + 1,\ j - j' \leq G_{s_2}[j] + 1\}$ The activation constraint is given by:

$$x_{i,j} \leq \sum_{(i',j') \in \text{Pred}(i,j)} x_{i',j'} + s_{i,j}, \qquad \forall (i, j) \in M.$$

2. *Starting position.* At most one starting pair is allowed:

$$\sum_{(i,j)\in M} s_{i,j} \leq 1$$

Moreover, a pair can be designated as the starting position only if it is selected:

$$s_{i,j} \leq x_{i,j}, \quad \forall (i,j) \in M.$$

3. *Conflict constraints (character ordering).* Two distinct pairs (i,j) and (i',j') are in conflict if they violate the character order:

$$\text{If } (i \leq i' \text{ and } j \geq j') \text{ or } (i \geq i' \text{ and } j \leq j'), \text{ then: } \quad x_{i,j} + x_{i',j'} \leq 1$$

4. *Auxiliary starting-position constraints.* If (i,j) is designated as the starting match, no selected match may precede it in both sequences:

$$\sum_{\substack{(i',j')\in M \\ i'<i,\, j'<j}} x_{i',j'} + s_{i,j} \leq 1, \qquad \forall (i,j) \in M.$$

Domains of the variables are given by

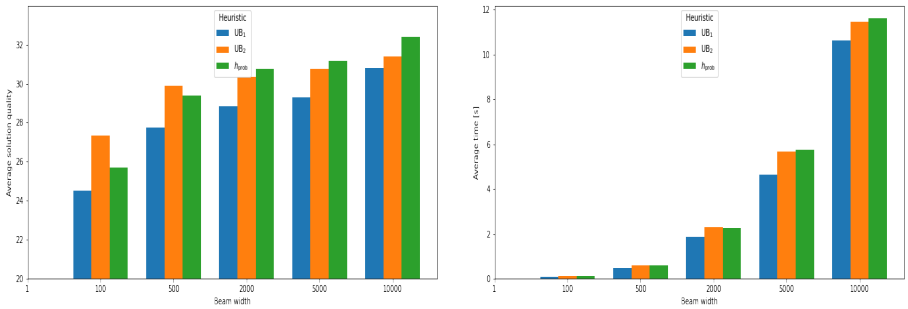
$$x_{i,j},\ s_{i,j} \in \{0,1\}, \quad \forall (i,j) \in M.$$

The model maximizes the number of selected matches forming a common subsequence while preserving character order and respecting the maximum allowed gaps between consecutive matches in both sequences.

5 Experimental Evaluation

We experimentally compare three heuristic approaches designed for arbitrary $m \geq 2$. Specifically, we consider: (i) Bs, a baseline beam search that performs a single IMSBS iteration with a large beam width; (ii) Imsbs-greedy, which fixes $\beta = 1$ and uses a large number of IMSBS iterations; and (iii) Imsbs, a tuned IMSBS configuration with an average runtime comparable to that of Bs. The algorithms were implemented in Python 3.11 and evaluated on the VEGA HPC system hosted at IZUM, Maribor. The cluster consists of 960 compute nodes equipped with AMD EPYC 7H12 CPUs operating at 2.35 GHz. All experiments were conducted in a single-threaded setting. For each combination of $n \in \{50, 100, 200, 500\}$, $m \in \{2, 3, 5, 10\}$, and $|\Sigma| \in \{2, 4\}$, 10 random problem instances were generated. First, m sequences of equal length were generated uniformly at random. The gap constraints were then generated by drawing each value $G_i(j)$, for $j = 1, \ldots, |s_i|$ and $i = 1, \ldots, m$, uniformly from $\mathcal{U}(\{\lfloor 0.5 \cdot |\Sigma| \rfloor, \ldots, \lfloor 1.5 \cdot |\Sigma| \rfloor\})$. Therefore, a total of *320 problem instances* were generated. The benchmark set is denoted as Random, freely available at `https://github.com/markodjukanovic90/IMSBS_for_VGLCSP/tree/main/instances`.

Parameter tuning. To tune the parameters of our algorithms, we employed a basic grid search for the most influential parameters: beam width in the beam search forward pass (β), and heuristic guidance $h \in \{\mathrm{UB}_1, \mathrm{UB}_2, h_{prob}\}$, comparing the average solution quality over all (320) problem instances. Less influential parameters are selected according to several preliminary runs. We fix $sources\_from\_R = 10$ and $beam\_iters = 100$ (beam iterations in IMSBS), and $h' = \mathrm{UB}_2$. For the baseline BS approach, a high beam width $\beta = 10,000$ and $h = h_{\mathrm{prob}}$ are utilized. Remaining parameters of the IMSBS approach received the following values: $h = \mathrm{UB}_2$ and $\beta = 500$ to keep comparable average runtime between IMSBS and the baseline BS approach. These values balance runtime relative to BS while maintaining a reasonable solution quality. The results of BS and IMSBS parameter tuning are displayed in Fig. 4–5. In the case of IMSBS-GREEDY approach ($\beta = 1$), we utilized $beam\_iters = 10,000$, while all other parameters remain as in the case of tuned IMSBS.

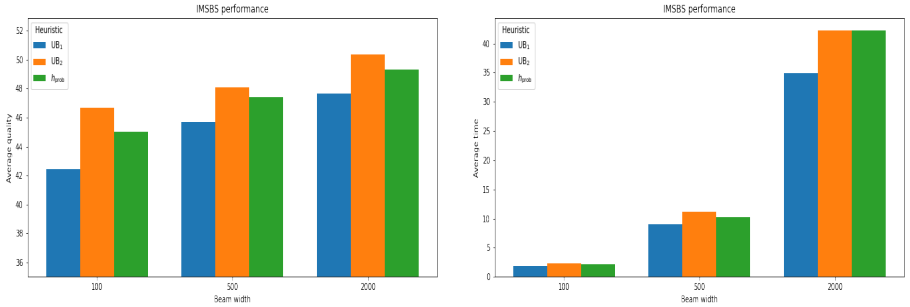


(a) Avg. quality over all instances of the RANDOM benchmark suite.

(b) Avg. runtime over all instances of the RANDOM benchmark suite.

Fig. 4: Results of BS parameter tuning on the RANDOM benchmark suite.

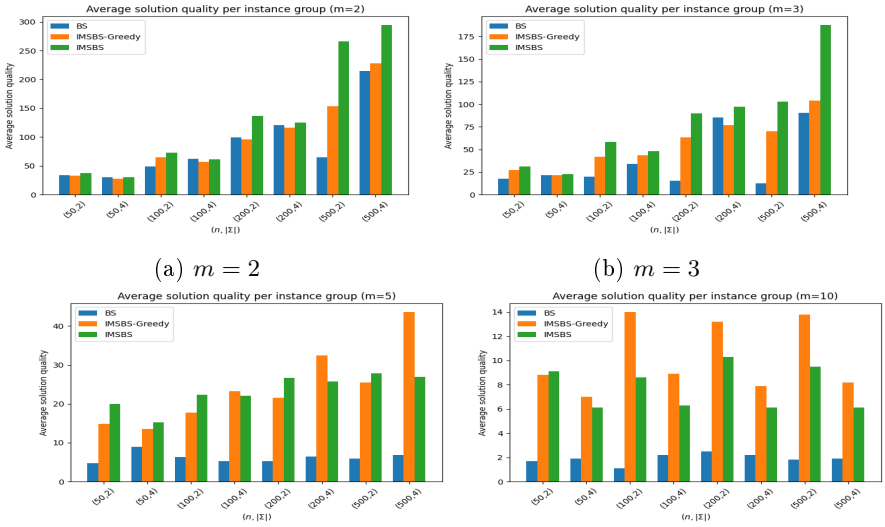
Numerical results for all three heuristic approaches are displayed in Fig. 6, reporting the average solution quality per instance group for each approach. Four plots are provided, one per each value of m. The IMSBS approach produces the highest average solution quality in 21 out of 32 instance groups, substantially outperforming the baseline BS approach, indicating that the selection of the right root node is essential in achieving high-quality solutions. BS performs better in just one case. Concerning the IMSBS-GREEDY, it surpasses the other two approaches in 10 (out of 32) cases, mostly for the instances with larger $m = 10$, having shorter final solutions where it turns out that frequently moving from one to another root and exploring various subregions of the search space is strategically more effective than investing the time for strengthening the search around a solution using a large β. Exploring additional root nodes improves the diversity of the search, which appears necessary in cases where expected solutions are short. This is exactly where IMSBS-GREEDY fits well. BS and IMSBS approaches



(a) Avg. solution quality of different IMSBS settings over all instances of the RANDOM benchmark suite.

(b) Avg. runtime of different IMSBS settings over all instances of the RANDOM benchmark suite.

Fig. 5: Results of IMSBS parameter tuning on the RANDOM benchmark suite.



(a) $m = 2$

(b) $m = 3$

(c) $m = 5$

(d) $m = 10$

Fig. 6: Average solution quality of the three approaches.

are tuned to exhibit comparable average runtimes. IMSBS-GREEDY has higher average runtimes for the instances with larger m due to a higher number of executed iterations ($beam\_iters = 10,000$ iterations allowed) which are computationally expensive when m is large. Detailed numerical results, instances, source code, and accompanying plots can be found in the GitHub repository at `https://github.com/markodjukanovic90/IMSBS_for_VGLCSP`.

5.1 Numerical Results for the $m = 2$ case

In this section, we compare the approaches from the literature that are specially designed for the case of two input sequences. The following approaches are compared:

- DP: the basic dynamic programming algorithm;
- DP-1: an advanced dynamic programming approach that uses Incremental Suffix Maximum Queries (ISMQ) with `Col` and `All` matrices to accelerate the basic DP algorithm;
- DP-2: an enhanced dynamic programming algorithm that handles ISMQ using a *dequeue* data structure for further speedup. All three DP approaches are from [11]. Note that the original paper proposes answering ISMQ using Union-Find operations; however, due to a lack of implementation details, our implementation employs a deque-based data structures.
- ILP: the proposed integer linear programming approach, motivated by the ILP model for the LCS problem [5]. Details on this approach are provided in Section 4.

Each algorithm was allocated 30 minutes of execution time. The ILP model was solved using the general-purpose solver CPLEX version 22.1.

The results of the four exact approaches for the VGLCS problem with two input sequences are reported in Table 1. It is organized as follows. The first three columns describe the characteristics of instance groups, over which the results are averaged (10 instances per group). The remaining columns report the performance (average objective value ($\overline{obj}$) and average execution time ($\overline{t}[s]$) in seconds) of the four algorithms: DP, DP-1, DP-2, and ILP. For each approach, both the average solution quality and the average execution time are provided.

Table 1: Results on the RANDOM benchmark set for $m = 2$: exact approaches.

Inst.			DP		DP-1		DP-2		ILP	
m	n	$\lvert\Sigma\rvert$	$\overline{obj}$	$\overline{t}[s]$	$\overline{obj}$	$\overline{t}[s]$	$\overline{obj}$	$\overline{t}[s]$	$\overline{obj}$	$\overline{t}[s]$
2	50	2	38.1	0.01	38.1	**0.01**	38.1	0.01	38.1	168.3
2	50	4	30.3	**0.01**	30.3	0.02	30.3	0.02	30.3	28.0
2	100	2	77.4	0.1	77.4	**0.03**	77.4	0.05	–	–
2	100	4	62.3	0.07	62.3	**0.06**	62.3	0.09	0.00	1800.0
2	200	2	156.4	0.75	156.4	**0.13**	156.4	0.16	–	–
2	200	4	127.2	0.59	127.2	**0.25**	127.2	0.32	–	–
2	500	2	395.9	13.57	395.9	**0.84**	395.9	1.05	–	–
2	500	4	317.2	10.18	317.2	**1.70**	317.2	2.1	–	–

Based on the numerical results reported in Table 1, the following conclusions can be drawn:

- All three dynamic programming approaches prove to be highly efficient, successfully solving all 80 problem instances to optimality.
- Runtimes of the DP methods to obtain optimal solutions are relatively short, in most cases remaining below 10 seconds.
- The ILP approach is capable of solving only the smallest instances with $n = 50$. Its execution times are approximately two orders of magnitude higher than those of the DP approaches. For larger instances, particularly those with $n = 200$, the ILP model fails to solve even the initial (root) relaxation.

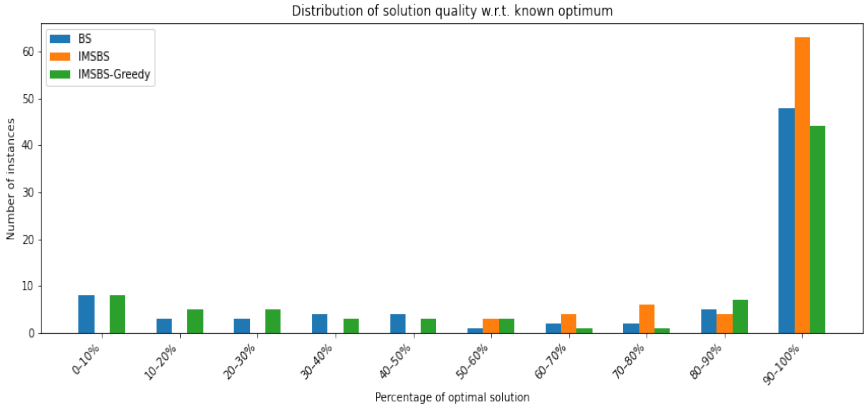
Concerning the efficiency of BS, IMSBS-GREEDY, and IMSBS on the instances solved optimally by dynamic programming ($m = 2$), Figure 7 presents the distribution of instances (y-axis) attaining specific ratios of solution quality relative to the known optimal solutions (x-axis).

In particular, IMSBS achieves near-optimal solutions in 63 out of 80 problem instances, whereas the corresponding numbers are significantly lower for the other two heuristic approaches. For instance, with $|\Sigma| = 4$ (and $m = 2$), the difference in solution quality is slightly in favor of IMSBS. However, for instances with a small alphabet ($|\Sigma| = 2$), IMSBS clearly outperforms the other two approaches.

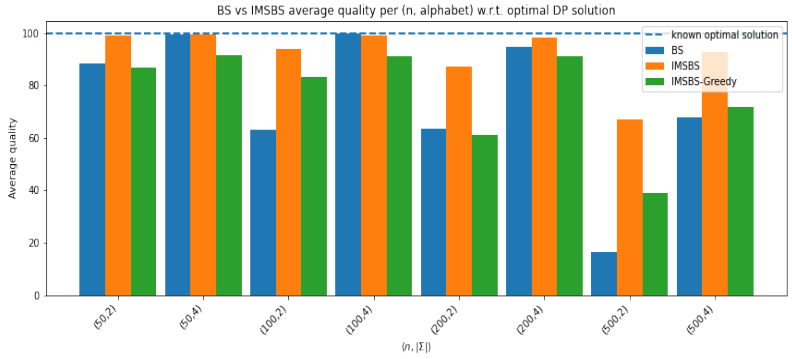
Nevertheless, for the group of instances with $n = 500$ and $|\Sigma| = 2$, there remains room for further improvement. This behavior can be explained by the known weakness in the LCS-based heuristic performance on instances characterized by small alphabet sizes and long sequences; therefore, alternative strategies should be explored, as mentioned in the future work section.

6 Conclusions and Future Work

This paper studies a generalized variable-gapped LCS problem. The model extends the classical LCS problem by constraining the allowable distances between consecutively matched characters, thereby capturing application-specific restrictions on the separation between matched residues or events. We proposed an Iterative Multi-Source Beam Search (IMSBS) that combines local exploration from promising root states with a global strategy for selecting candidate roots based on information collected during the search. Experimental results indicate that IMSBS generally produces higher-quality solutions than the baseline beam search. When shorter solutions are expected, frequent transitions between feasible rooted subgraphs are more effective. By contrast, for instances with fewer sequences, a larger beam width and more IMSBS iterations are needed to obtain high-quality solutions. Future work will focus on designing more sophisticated heuristics that incorporate gap constraints into the scoring function and on optimizing IMSBS so that it can efficiently reuse previously explored regions, and testing the approaches on real-world biological instances.



(a) Percentage distribution of relative solution quality for each heuristic approach with respect to the optimal solutions.



(b) Relative solution quality achieved by the heuristic approaches compared to the optimal solutions.

Fig. 7: Number of instances for which Bs and Imsbs achieve a specific relative ratio of the obtained solution quality with respect to the known optimal solutions.

Acknowledgments

This publication is co-funded by the European Union's Horizon Europe research and innovation program under the Marie Sklodowska-Curie COFUND Postdoctoral Programme (grant agreement No. 101081355 – SMASH), and by the Republic of Slovenia and the European Union through the European Regional Development Fund. Views and opinions expressed are those of the authors only and do not necessarily reflect those of the European Union or the European Research Executive Agency (REA). Neither the European Union nor the REA can be held responsible for them.

The authors gratefully acknowledge the SLING consortium for funding this research by providing computing resources of the HPC Vega at the Institute of Information Science (www.izum.si).

References

1. Adamson, D., Kosche, M., Koß, T., Manea, F., Siemer, S.: Longest common subsequence with gap constraints. In: International Conference on Combinatorics on Words. pp. 60–76. Springer (2023)
2. Adi, S.S., Braga, M.D., Fernandes, C.G., Ferreira, C.E., Martinez, F.V., Sagot, M.F., Stefanes, M.A., Tjandraatmadja, C., Wakabayashi, Y.: Repetition-free longest common subsequence. Discrete Applied Mathematics **158**(12), 1315–1324 (2010)
3. Bergroth, L., Hakonen, H., Raita, T.: A survey of longest common subsequence algorithms. In: Proceedings Seventh International Symposium on String Processing and Information Retrieval. SPIRE 2000. pp. 39–48. IEEE (2000)
4. Blum, C., Djukanovic, M., Santini, A., Jiang, H., Li, C.M., Manyà, F., Raidl, G.R.: Solving longest common subsequence problems via a transformation to the maximum clique problem. Computers & Operations Research **125**, 105089 (2021)
5. Blum, C., Festa, P.: Metaheuristics for String Problems in Bio-informatics, vol. 6. John Wiley & Sons (2016)
6. Djukanovic, M., Raidl, G., Blum, C.: Finding longest common subsequences: New anytime A* search results. Applied Soft Computing **95**, 106499 (2020)
7. Farhana, E., Rahman, M.S.: Constrained sequence analysis algorithms in computational biology. Information Sciences **295**, 247–257 (2015)
8. Jiang, T., Lin, G., Ma, B., Zhang, K.: The longest common subsequence problem for arc-annotated sequences. Journal of Discrete Algorithms **2**(2), 257–270 (2004)
9. Lainscsek, C., Sejnowski, T.J.: Delay differential analysis of time series. Neural computation **27**(3), 594–614 (2015)
10. Mousavi, S.R., Tabataba, F.: An improved algorithm for the longest common subsequence problem. Computers & Operations Research **39**(3), 512–520 (2012)
11. Penga, Y.H., Yangb, C.B.: The longest common subsequence problem with variable gapped constraints. In: Proceedings of the 28th Workshop on Combinatorial Mathematics and Computation Theory, Penghu, Taiwan. pp. 17–23 (2011)